

Konzeption und Implementierung eines GitHub-basierten Branching-, Review- und Projektmanagementprozesses mit KI-gestützter Code-Compliance

T4_2000

Studiengang Elektrotechnik

Duale Hochschule Baden-Württemberg Ravensburg
Campus Friedrichshafen

Florian Germann

Eingereicht am: 23.04.2026

Matrikelnummer, Kurs: 7582635, TEA24

Unternehmen: fpt Systems GmbH, 88279, Amtzell

Betreuer im Unternehmen: Simon Würzer

Inhalt

Erklärung zum Einsatz von KI-gestützten Werkzeugen	1
1 Einleitung	3
2 Problematik	5
3 Lösungsansatz	7
4 Aktueller Stand Coding Conventions	9
4.1 Industriestandards	9
4.1.1 GitHub-Flow	9
4.1.2 Git-Flow	11
4.2 Richtlinien E-SW	13
4.3 Fazit	15
5 Prüfung der Coding-Conventions	16
5.1 Aktueller Stand bei fpt	16
5.2 Testablauf	17
5.3 Probleme und potenzielle Lösungen	18
5.4 Feedback und andere Verbesserung	22
6 Testen Branching und Code-Review	23
6.1 Umsetzung	24
6.1.1 Branching Strategien	24
6.1.2 Copilot Code-Review	24
7 Implementierung ZenHub	25
7.1 Aktueller Stand bei E-SW	25
8 Fazit	26
A Literatur	27
B Glossar	28



Erklärung zum Einsatz von KI-gestützten Werkzeugen

Im Rahmen dieser Projektarbeit wurden KI-gestützte Werkzeuge ausschließlich als unterstützende Hilfsmittel eingesetzt. Die folgende Tabelle gibt einen Überblick über die genutzten Systeme und deren konkreten Verwendungszweck:

Werkzeug	Anbieter	Verwendungszweck
GitHub Copilot	Microsoft / GitHub	Automatisiertes Code-Review anhand der Coding-Conventions bei Pull Requests
GitHub Copilot Chat	Microsoft / GitHub	Analyse von Review-Problemen, Recherche zu Lösungsansätzen, Überprüfung von Rechtschreibung
GitHub Copilot Agent	Microsoft / GitHub	Tiefergehende Repository-Analyse zur Ursachenforschung bei Kommentarunterdrückung
Microsoft Copilot „Prompt Coach,,	Microsoft	Unterstützung bei der KI-kompatiblen Überarbeitung und Präzisierung der Coding-Conventions

Tabelle 1 – Eingesetzte KI-Werkzeuge

Alle durch KI-Werkzeuge generierten Ausgaben wurden vom Autor inhaltlich geprüft, bewertet und erst nach kritischer Sichtung übernommen. Eine automatische oder ungeprüfte Übernahme von KI-Inhalten erfolgte nicht. Die inhaltliche Verantwortung für diese Arbeit liegt vollständig beim Autor.

KI-Werkzeuge wurden nicht zur Erstellung des Fließtextes dieser Arbeit eingesetzt, sondern ausschließlich zur Prüfung auf Rechtschreibfehler.



Einleitung

Effiziente und strukturierte Quellcodeverwaltung ist zentral für moderne Softwareentwicklung. In verteilten Teams, in denen parallel an verschiedenen Komponenten gearbeitet wird, gewinnt die Organisation von Versionsständen hohe Bedeutung. Git-basierte Plattformen wie GitHub bieten hierfür leistungsfähige Lösungen.

GitHub ist eine webbasierte Plattform zur zentralen Speicherung, Verwaltung und kollaborativen Weiterentwicklung von Softwareprojekten. Durch die Git-Versionskontrolle werden Änderungen nachvollziehbar dokumentiert und frühere Stände wiederherstellbar. Dies erhöht Transparenz und Nachverfolgbarkeit in komplexen Projekten.

Ein zentrales Konzept ist der Branch. Branches sind isolierte Entwicklungszweige, mit denen neue Funktionen, Fehlerbehebungen und Experimente von der Hauptlinie getrennt durchgeführt werden können. Änderungen werden erst nach Prüfung zurückgeführt. So sind parallele Arbeiten möglich ohne gegenseitige Störungen.

Durch diese Kombination aus Versionskontrolle, klarer Nachvollziehbarkeit und parallelen Entwicklungszweigen bietet GitHub eine strukturierte Grundlage für moderne Softwareentwicklung in Teams verschiedener Größe.

Die Effektivität von GitHub ist abhängig von der Wahl einer geeigneten Branching-Strategie und dem richtigen Team Management. Branching-Regeln definieren, wie Features, Bugfixes und Releases organisiert und integriert werden. Sie sind technisch und prozessual relevant für Koordination und Qualitätssicherung.

Die Effektivität hängt jedoch nicht nur von einer geeigneten Branching-Strategie ab, sondern auch von einer strukturierten Aufgabenplanung im Team. Spezialisierte Projektmanagement-Tools helfen dabei, Arbeitsabläufe zu strukturieren, Verantwortlichkeiten klar zuzuordnen und den Fortschritt jederzeit nachvollziehbar zu machen.

Durch die enge Verzahnung mit Entwicklungsprozessen, automatisierten Workflows und übersichtlichen Boards entsteht ein gemeinsames Verständnis darüber, welche Aufgaben als Nächstes anstehen und wo mögliche Engpässe liegen. Solche Lösungen fördern nicht nur die Zusammenarbeit, sondern erhöhen auch die Qualität und Geschwindigkeit der gesamten Produktentwicklung.



Problematik

BRANCHING

In der Abteilung **O-SW** fehlt bislang eine einheitliche und verbindliche Vorgehensweise für den Umgang mit Branches in GitHub. Obwohl die Richtlinien aus der **E-SW** bereits für interne Bibliotheken und Template-Projekte übernommen wurden, zeigt sich, dass diese Vorgaben unter Umständen nicht ohne Weiteres auf die Arbeit an den Anlagen übertragbar sind. Die dortigen Entwicklungsabläufe unterscheiden sich strukturell und inhaltlich, wodurch Unklarheiten und Inkonsistenzen im praktischen Einsatz entstehen können.

Diese fehlende Passgenauigkeit führt dazu, dass Branching-Entscheidungen individuell und uneinheitlich getroffen werden. Dadurch entstehen Risiken wie unklare Versionsstände, erschwerte Nachvollziehbarkeit von Änderungen und ein erhöhtes Konfliktpotenzial beim Zusammenführen von Code. Gleichzeitig erschwert die heterogene Anwendung bestehender Richtlinien eine effiziente Zusammenarbeit innerhalb des Teams und beeinträchtigt die Qualitätssicherung im Entwicklungsprozess.

MANAGEMENT

Zum aktuellen Zeitpunkt besteht in der **O-SW** die Problematik, dass keine Team-Management-Software vorhanden ist und die Release-Planung hauptsächlich über mündliche Abstimmungen und E-Mail-Ketten erfolgt. Zudem entstehen regelmäßig Schwierigkeiten, da neue Versionen von **FlexOp** für die **O-SW** teilweise kurzfristig auftreten und entsprechende Vorbereitungen fehlen.

CODE-REVIEW

Außerdem besteht trotz klar formulierter Coding-Conventions die Problematik, dass falsch benannte Variablen oder unzureichend dokumentierter Code in die Projekte gelangen. Dies ist darauf zurückzuführen, dass die Einhaltung der Richtlinien bislang nur eingeschränkt geprüft wird. Ein wesentlicher Grund dafür ist, dass für [TwinCat](#) keine konventionellen Linter¹ oder vergleichbare Standardwerkzeuge verfügbar sind.

Aus diesen drei Problemfeldern ergibt sich der Bedarf nach einem einheitlichen, technisch umsetzbaren und abteilungsweit akzeptierten Lösungsansatz.

¹Ein Linter prüft Code automatisch auf Fehler und Stilabweichungen.



Lösungsansatz

BRANCHING

Die Abteilung **E-SW** hat bereits erfolgreich Branching-Strategien etabliert. Diese Richtlinien werden in der **O-SW** bereits für interne Bibliotheken und Template-Projekte genutzt und sollen im nächsten Schritt auf die Arbeit an den Anlagen erweitert werden.

Dafür werden die bestehenden Strategien systematisch auf ihre Eignung für die spezifischen Abläufe der **O-SW** geprüft und zu konsistenten, unternehmensweit gültigen Richtlinien weiterentwickelt.

MANAGEMENT

Für die unzureichend strukturierten Management-Abläufe in der **O-SW** existiert in der **fpt Systems GmbH** bereits ein erprobter Ansatz. In der Abteilung **E-SW** wird seit einiger Zeit mit der Team-Management-Software „ZenHub“ gearbeitet. Sie dient dazu, anstehende Aufgaben nach Priorität zu ordnen, Teammitgliedern zuzuweisen und die Releases von **FlexOp** sowie weiterer Hochsprachen-Projekte zu planen.

„ZenHub“ soll daher ebenfalls in der Abteilung **O-SW** eingesetzt werden, um die Weiterentwicklung der Standardbibliotheken sowie die Aktualisierung der Template-Projekte für die SPS-Programmierung zu strukturieren. Zusätzlich soll damit die Release-Koordinierung mit der **E-SW** verbessert werden.

CODE-REVIEW

Zur Verbesserung der Code-Qualität wird ein flexiblerer Ansatz als bei klassischen Lintern verfolgt. In Abstimmung mit dem KI-Experten der E-SW und dem für Richtlinien verantwortlichen Mitarbeiter der O-SW Robin Hettel, wurde daher die Einführung von GitHub-Copilot-gestützten Code-Reviews vorgeschlagen. Dieser Ansatz lässt sich auf projektspezifische Anforderungen und Vorgaben anpassen und unterstützt so eine effizientere Qualitätssicherung.

FAZIT

Um die korrekte und effiziente Nutzung dieser Verbesserungen zu gewährleisten, müssen nach der Implementierung alle Mitarbeitenden über die Änderungen in Kenntnis gesetzt und entsprechend geschult werden. Darüber hinaus sind die erarbeiteten Abläufe zentral zu dokumentieren und zugänglich zu machen, um die nachhaltige Anwendung sicherzustellen.

Alle Maßnahmen dienen der Steigerung der Arbeits- und Code-Qualität in der fpt Systems GmbH.

4

Aktueller Stand Coding Conventions

4.1 Industriestandards

In der Softwareindustrie haben sich verschiedene Branching-Standards etabliert, die seit vielen Jahren eingesetzt werden. Im Folgenden werden die zwei bekanntesten Modelle erläutert und auf ihre Anwendbarkeit für die vorliegende Situation analysiert.

4.1.1 GitHub-Flow

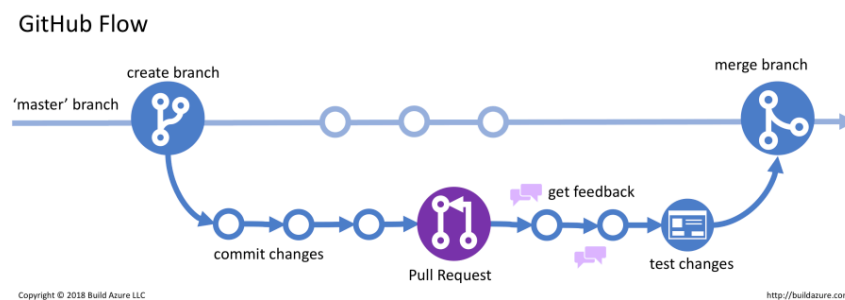


Abbildung 1 – GitHub Flow

GitHub-Flow ist ein leichtgewichtiges, kontinuierlich ausgerichtetes Branching-Modell, das speziell für Teams entwickelt wurde, die schnell und flexibel deployen möchten. Es setzt auf kurze Feature-Branche, häufige Pull Requests und automatisierte Tests, um Änderungen zügig in den Main-Branch zu integrieren. Besonders verbreitet ist GitHub-Flow in modernen DevOps-Umgebungen und Projekten, in denen kontinu-

ierliche Auslieferung und hohe Release-Geschwindigkeit im Vordergrund stehen, wie z. B. bei Web-Apps.

Branch	Beschreibung
main(master)	Enthält fertige Releases Stabil - Änderungen nur aus feature/* per PR
feature/*	Entwicklungsbranch für einzelne Features oder Fixes werden direkt von main erstellt nach Fertigstellung direkt PR in main

In Tabelle 2 sind die einzelnen Branches und ihre Funktion beschrieben; Abbildung 1 visualisiert den Ablauf ergänzend. Damit wird deutlich, dass diese Strategie insbesondere für schnelle Feature-Releases geeignet ist. Dies gilt jedoch nur unter bestimmten Rahmenbedingungen, da die Eignung stets von den spezifischen Anforderungen der Organisation abhängt.

VORTEILE

- Schnelle Iteration
- Geringe Komplexität
- Ideal für kontinuierliche Integration

NACHTEILE

- Wenig Struktur für große Releases
- Risiko eines instabilen Main-Branche bei häufigem Mergen

FAZIT

Zusammenfassend bietet GitHub-Flow eine einfache und agile Branching-Strategie, die sich ideal für Teams eignet, die kontinuierliche Integration und schnelle Feature-Releases priorisieren. Durch kurze Feature-Branche und direkte Mergen in den Main-Branch fördert es Flexibilität und reduziert Overhead. Im Kontext der O-SW-Abteilung, die auf strukturierte Meilenstein-Releases setzt, zeigt sich jedoch eine Schwäche: Die fehlende Isolation von Release-Vorbereitungen kann zu Instabilitäten führen und die Qualitätssicherung erschweren. Daher ist GitHub-Flow weniger geeig-

net als komplexere Modelle wie Git-Flow, die eine klarere Trennung von Entwicklungs- und Release-Phasen bieten.

GitHub-Flow zeigt somit Grenzen für die O-SW. Im Folgenden wird die Alternative Git-Flow analysiert, um festzustellen, ob sie besser zum Kontext passt.

4.1.2 Git-Flow

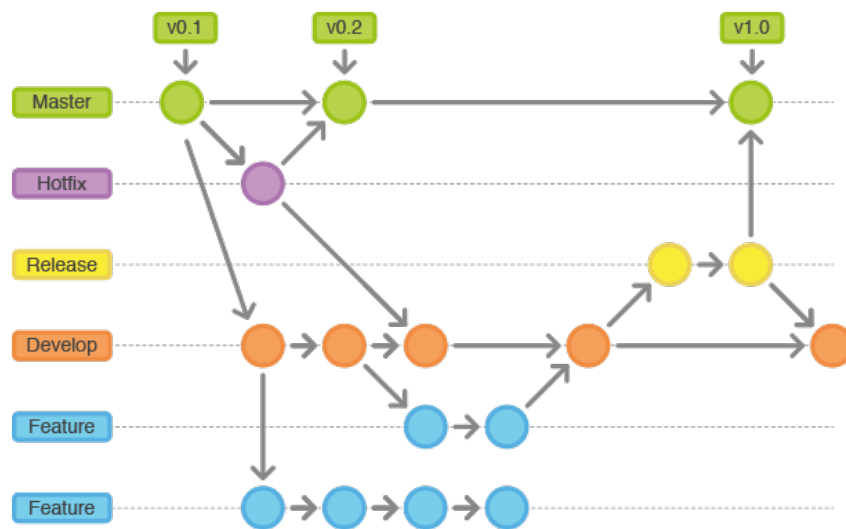


Abbildung 2 – Git Flow

Git-Flow ist ein strukturiertes Branching-Modell für Git, das Teams hilft, Entwicklungsprozesse klar zu organisieren und gleichzeitig stabile Releases sicherzustellen. Es wurde 2010 von Vincent Driessen entwickelt und ist besonders in Projekten verbreitet, in denen mehrere Features, Hotfixes und Versionen parallel entstehen.

Das Modell umfasst mehrere langlebige Branches mit klar definierten Regeln. In Abbildung 2 wird dargestellt, wie sich diese Branches zueinander verhalten und in welchen Branches tatsächlich entwickelt wird. Ergänzend werden in Tabelle 3 die einzelnen Branches beziehungsweise Branch-Gruppen beschrieben.

Branch	Beschreibung
main(master)	Enthält fertige Releases Stabil - Änderungen nur aus release/* per PR Änderungen in main müssen zurück in develop fließen.
develop	Grundlage aller Entwicklungsbranches Stabil - Änderungen nur als PR
feature / *	Entwicklungsbranches neuer Features In diesen wird gearbeitet Strukturierung über Subbranches möglich
bugfix / *	Entwicklungsbranch für Bugfixes Wird von develop abgespalten um Bugs zu entfernen
release / *	Vorbereitung für den Release Wird von develop abgespalten Änderungen fließen wieder in develop z. B. Erstellung von Release Notes
hotfix / *	Hotfix von main Änderungen fließen auch in develop

Dieses System eignet sich hervorragend für strukturierte Abläufe mit klar definierten, umfassenden und sequenziellen Releases. Es ist jedoch weniger geeignet für die Entwicklung von „Rolling Releases“, da die Zeitspanne bis zur Veröffentlichung zu lang ist. Da die Projekte in der [O-SW](#) Releases nur an Projektmeilensteinen vorsehen, ist dieses System am besten geeignet.

Branch	Beschreibung
main	Für Releasevorbereitung Stabil - Änderungen nur aus „develop“ per PR Änderungen in main müssen zurück in develop fließen.
develop	Grundlage aller Entwicklungsbranches Stabil - Änderungen nur als PR
feature / *	Entwicklungsbranch neuer Features Verknüpfung mit ZenHub Epics über ID möglich Beispiel: feature/Fx.FlexOP-1640_SSI-Packagerstellung SubFeature (Task-Ebene): feature/Fx.FlexOP-1640_SSI-Packagerstellung/FPT.SSI-51_Externe-Steuerungsconfig
bugfix / *	Entwicklungsbranch für Bugfixes Verknüpfung mit ZenHub Bugs über ID möglich Beispiel: bugfix/Fx.FlexOP-1234
release / *	Releases von main Kann auch über Tag gelöst werden, sollte aber als Branch gezogen werden, sobald ein Hotfix nötig wird. Beispiel: release/Fx.FlexOP_v4.5.5.x
hotfix / *	Hotfix von Release-Branch Sollte wieder zurück in develop fließen Verknüpfung mit ZenHub Bug über ID Beispiel: hotfix/Fx.FlexOP_v4.5.5.12_2547

Dieses Word-Dokument enthält in Abbildung 3 und Tabelle 4 die Kernelemente einer angepassten Git-Flow-Strategie. Die Strategie unterscheidet sich dadurch, dass

Releases nicht in separaten release/*-Branches vorbereitet werden, sondern in main, und dann über einen release/*-Branch veröffentlicht werden. Die übrigen Vor- und Nachteile entsprechen denen aus Abschnitt 4.1.2.

4.3 Fazit

Auf Basis von Gesprächen mit Robin Hettel wurde festgestellt, dass die Implementierung von Git-Flow sowohl zweckmäßig als auch gewünscht ist. Dies liegt daran, dass Git-Flow für wenige, größere Releases besser geeignet ist als GitHub-Flow. Darüber hinaus ist der in Abschnitt 4.2 genannte Vorteil für den vorliegenden Kontext weniger relevant, da Releases ausschließlich an Projektmeilensteinen erfolgen und der aktuelle Code-Stand somit die primäre Anforderung darstellt.



Prüfung der Coding-Conventions

GitHub Copilot kann weit mehr als nur Code vervollständigen: Das System unterstützt Entwickler auch dabei, bestehende Coding-Conventions automatisiert zu berücksichtigen. Während des Schreibens analysiert Copilot den Kontext, erkennt typische Muster und orientiert sich an projektinternen Standards. Dadurch entstehen Vorschläge, die nicht nur funktional, sondern auch stilistisch konsistent sind. Teams profitieren so von einer gleichmäßig hohen Codequalität, weniger manuellen Korrekturen und einem deutlich effizienteren Review-Prozess.

5.1 Aktueller Stand bei fpt

Bei fpt befindet sich der Einsatz von Copilot für Code-Reviews aktuell noch in der Erprobungsphase. Hierfür wurde ein separates Test-Repository erstellt, um keine kritischen Bereiche zu beeinflussen. Da Copilot bei Reviews nur auf das jeweilige Repository zugreifen kann, müssen die Coding-Conventions im `.github`-Ordner hinterlegt werden. Dies erfolgt über `.md`-Dateien, die GitHub nativ in gerenderter Form anzeigt. Um sicherzustellen, dass alle Repositories konsistent aktualisiert werden und Änderungen nicht manuell organisationsweit verteilt werden müssen, wurde das in Abbildung 4 dargestellte System implementiert. Jeder Block repräsentiert ein Repository, die Pfeile symbolisieren GitHub Actions.

Bei Änderungen der Coding-Conventions wird im Übersichts-Repository eine GitHub-Aktion ausgeführt, die die aktuellen `.md`-Dateien an Bibliotheken und Template-Projekte verteilt. Da in den Aktionen die Ziel-Repositories explizit angegeben werden

müssen, lassen sich Projekt-Repositories auf diesem Weg nicht direkt anpassen, weil diese fortlaufend neu erstellt werden. Stattdessen werden im Template-Projekt Aktionen hinterlegt, die in allen abgeleiteten Projekten verfügbar sind und die aktuellen Conventions aus dem Übersichts-Repository abrufen. Auf diese Weise sind die aktuellen Conventions in allen relevanten Repositories verfügbar.

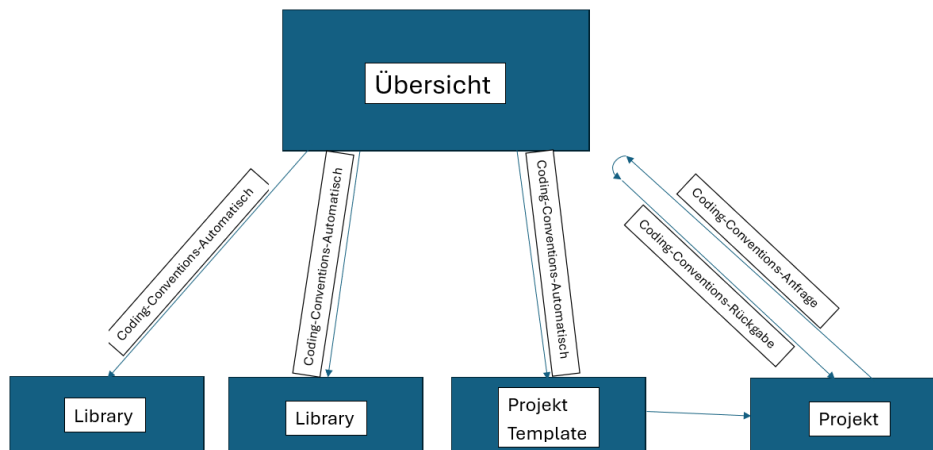


Abbildung 4 – Coding-Convention Verteilung

Diese Aktionen sind bereits in allen O-SW-Repositories vorhanden, die keine Projekt-Repositories sind. Diese wurden intensiv auf Fehler getestet. Auch die Funktion, dass neue Projekte diese Aktionen automatisch erhalten, ist bereits implementiert.

Des Weiteren wurde im Test-Repository bereits implementiert, dass Copilot bei jeder Pull Request auf main automatisch alle Commits hinsichtlich der Coding-Conventions prüft und entsprechende Verbesserungsvorschläge liefert. Zur Überprüfung der Zuverlässigkeit dieser Funktion wurden bewusst Fehler eingebaut. Aus den resultierenden Copilot-Antworten auf die Pull Requests konnten mehrere Problemfelder abgeleitet werden. Diese sollen im Folgenden aufgelistet, erläutert und potenziell gelöst werden.

5.2 Testablauf

Zur Evaluierung der Zuverlässigkeit von GitHub Copilot wurde im Test-Repository ein separater Branch angelegt, in den gezielt Fehler eingebaut wurden. Diese repräsentieren häufig auftretende Programmierfehler und dienen der Identifikation typischer Schwachstellen. Anschließend wurde jeweils ein Pull Request in main erstellt, um die konfigurierte Code-Review-Richtlinie zu aktivieren. Copilot analysierte daraufhin die Änderungen und verfasste entsprechende Kommentare zum Pull Request.

Da Copilot die Pull-Request-Beschreibung und weitere Kommentare verarbeitet, wurde die Checkliste mit den absichtlich eingebauten Fehlern jeweils erst nach dem Review ergänzt, um den Entwicklungsverlauf nachvollziehbar zu dokumentieren.

Wenn Änderungen an den Coding-Conventions oder Copilot-Anweisungen vorgenommen wurden, wurde der vorherige Pull Request geschlossen und ein neuer erstellt, um eine kontrollierte und vergleichbare Testumgebung zu gewährleisten.

5.3 Probleme und potenzielle Lösungen

GRUNDPROBLEM

In den Code-Reviews durch Copilot zeigte sich zunächst, dass die Gründlichkeit nicht ausreichte. Nach Recherchen und Konsultation von GitHub Copilot Chat wurde deutlich, dass die Coding-Conventions vermutlich nicht präzise genug formuliert waren. Deshalb wurden die vollständigen Kapitel „[TwinCat](#)“ und „Allgemein“ überarbeitet. Hierzu wurde der Microsoft-Copilot-Agent „Prompt Coach“ eingesetzt, um die Regeln unter Berücksichtigung des vorhandenen Kontexts KI-kompatibler zu formulieren. Die Überarbeitung umfasste vor allem eine sprachliche Präzisierung sowie die Ergänzung separater Copilot-Anweisungen.

Die durch die KI generierten Vorschläge wurden von mehreren Personen inhaltlich geprüft und nur nach sorgfältiger Bewertung übernommen. Eine ungeprüfte oder automatische Übernahme der KI-Ausgaben erfolgte nicht.

Zusätzlich wurde jeder Regel eine eindeutige ID zugeordnet, die von GitHub Copilot referenziert werden kann. Die Struktur folgt dem Muster Kapitelnummer-Fehlernummer im Kapitel, zum Beispiel „3.2.7-2“.

Nach diesen Anpassungen verbesserte sich die Qualität der Reviews deutlich. Dies zeigte sich daran, dass die Referenzierung von Kapiteln beziehungsweise Regel-IDs konsistent funktionierte und identifizierte Probleme zumindest als unterdrückte Kommentare erschienen.

PROBLEM KOMMENTARUNTERDRÜCKUNG

Das zentrale Restproblem bestand darin, dass Copilot Benennungsregelverstöße für Variablentypen innerhalb von Structs nur eingeschränkt zuverlässig erkannte. Insbesondere die Prüfung des Präfixes `stc_` für Struct-Typ-Variablen war betroffen. In der

Praxis äußerte sich dies so, dass Verstöße zwar erkannt, die zugehörigen Kommentare jedoch unterdrückt wurden. Das Problem wurde reproduziert, indem im Testcode gezielt das erforderliche Präfix bei einem Variablentyp in einer Struct entfernt wurde. Zur Lösung wurden die relevanten Auszüge aus den Coding-Conventions in Abbildung 5 in den Kontextdialog übernommen. Nach Beschreibung des konkreten Problems wurde der Zusatz für die Coding-Conventions erstellt, der in Listing 1 dargestellt ist.

****3.2.5-1 (Major)****
 Der Datentyp JEDER Variable MUSS durch ein eindeutiges Präfix gekennzeichnet sein.

IEC-Datentyp	Präfix
-----	-----
BOOL	b_
BIT	bit_
BYTE	by_
WORD	w_
DWORD	dw_
LWORD	lw_
INT	i_
UINT	ui_
SINT	si_
USINT	usi_
DINT	di_
UDINT	udi_
LINT	li_
ULINT	uli_
REAL	r_
LREAL	lr_
TIME	t_
LTIME	lt_
DATE	d_
TIME_OF_DAY	tod_
STRING	str_
PVOID	pvo_
ARRAY	a_
STRUCT	stc_
ENUM	e_
UNION	u_
ANY	any_
REFERENCE TO	ref_
POINTER TO	ptr_

Abbildung 5 – Coding-Convention Variablen Präfixe

```
1  ### Zusatz: Referenzierung der Datentyp-Präfixe bei  
2  Benennungsverstößen (Mandatory)  
3  
4  Bei Benennungsverstößen, die Datentyp-Präfixe betreffen,  
5  MUSS als fachliche Grundlage **Tabelle 3.2.5 (Datentyp-Präfixe)**  
6  herangezogen werden.  
7  
8  Befunde MÜSSEN:  
9  - das **konkret erwartete Präfix** nennen (z.B. `b_`, `i_`, `di_`)  
10 - optional explizit auf **Tabelle 3.2.5** verweisen  
11  
12 Befunde DÜRFEN NICHT:  
13 - Datentypen semantisch interpretieren (z.B. „BOOL-Präfix“)  
14 - implizite Ableitungen ohne Tabellenreferenz enthalten  
15  
16 Zulässig:  
17 - „Feldname beginnt nicht mit dem erwarteten Präfix `b_` gemäß Tabelle 3.2.5.“  
18 - „Bezeichner beginnt nicht mit dem erforderlichen Präfix `e_` (siehe Tabelle  
19 3.2.5).“
```

Listing 1 – Klausel

Nach dieser Änderung erkannte GitHub Copilot die Regelverstöße zuverlässig. Allerdings erschienen die Hinweise weiterhin als „suppressed due to uncertainty“. Zur Analyse wurde die GitHub-Copilot-Funktion „Agents“ genutzt. Diese kann das gesamte Repository einschließlich früherer Pull Requests als Kontext einbeziehen und liefert dadurch fundiertere Analysen. Die Auswertung zeigte, dass die Kommentare von GitHub selbst unterdrückt wurden, weil sie eine umbenannte Datei referenzierten. Da Git-Umbenennungen in Diffs häufig als Löschen und Neu-Anlegen dargestellt werden, stuft GitHub solche Befunde schneller als unsicher ein, weil:

- Die gesamte Datei als „neu hinzugefügt,“ erscheint
- GitHub keine stabile Diff-Kontext-Verknüpfung für umbenannte Dateien aufbauen kann
- Das Umbenennen an sich als intentionale Änderung gilt. Copilot widerspricht dem PR-Intent

Daraus folgt: Copilot erkennt den Verstoß korrekt, GitHub unterdrückt den Kommentar jedoch im Anschluss.

Die durch die Agent gelieferten Analysen wurden manuell überprüft und mit den tatsächlichen Repository-Änderungen abgeglichen, um Fehlinterpretationen auszuschließen.

Als Gegenmaßnahme schlug der Agent vor, die in Listing 2 gezeigte Anweisung in die GitHub-Copilot-Instruktionen aufzunehmen.

```
1 ### Pflicht-Strategie für Violations in umbenannten DUT-Dateien  
2 (Mandatory)  
3 **3.2.12-2 (Mandatory Review Behavior)**  
4  
5 Wenn ein STRUCT- oder ENUM-Typ in einer **umbenannten** DUT-Datei  
6 deklariert wird  
7 (erkennbar daran, dass der alte Dateiname und der neue Dateiname im  
8 \.plcproj`-Diff  
9 sichtbar sind), MUSS Copilot den Typ-Benennungsverstoß auf **zwei** Arten  
10 melden:  
11  
12 1. An der Verwendungsstelle in einer **nicht umbenannten, modifizierten  
13 Datei**  
14 (z.B. `stc_FxControlPar.TcDUT`, `FxGroupManagement_V12.TcPOU`), sofern  
15 der  
16 Typ dort als neue Version des umbenannten Typs referenziert wird.  
17 Die Evidence MUSS die Zeile enthalten, die den Typnamen ohne korrektes Präfix  
18 zeigt.  
19 Der Befund MUSS explizit darauf hinweisen, dass die TYPE-Deklaration selbst  
verletzt ist.  
2  
2. In der \.plcproj`-Datei an der Zeile, die den neuen (prefix-losen) DUT-  
Dateinamen  
enthält - sofern diese Zeile im Diff als geänderter `+`-Eintrag erscheint.  
  
**Copilot DARF NICHT** als einzigen Fundort die umbenannte DUT-Datei selbst  
angeben,  
da GitHub diese Kommentare als low-confidence unterdrückt.
```

Listing 2 – Anti suppression Klausel

Damit werden Verstöße nicht nur an der Deklaration, sondern zusätzlich an der ersten Verwendung in einer nicht umbenannten Datei gemeldet. Dadurch sinkt die Wahrscheinlichkeit einer Unterdrückung durch GitHub. Nach dem Testen dieses Vorschlags wurden die ursprünglichen Kommentare zwar weiterhin teilweise unterdrückt, die Verstöße erschienen jedoch zusätzlich an der ersten Verwendung der betroffenen Variable als Fehler. Da diese Fundstellen nicht in umbenannten Dateien lagen, wurden die entsprechenden Meldungen nicht unterdrückt.

5.4 Feedback und andere Verbesserung

Nachdem die Verbesserungen auf dem Test-Repository implementiert wurden, fand ein Gespräch mit Ralf Scheyerle statt, welcher die Änderungen prüfte. Hierbei merkte er einige Dinge an den Instructions an. Zum einen seien sie ziemlich eng formuliert, sodass der Copilot nur als Reviewer funktionieren kann, was die Nutzung von ihm als Coding Agent in der Zukunft erschweren würde. Da diese Funktion ein großer Teil der effektiven Nutzung von KI ausmacht, empfahl er hier, die Anweisungen abzuändern, um diese Möglichkeit zu erhalten. Hierbei muss aber eine Balance gefunden werden, zwischen Freiheit und richtiger Funktion des Modells.

Im Nachgang dieses Gesprächs wurden zum einen die Instructions so überarbeitet, dass sie den Copilot nichtmehr zu stark einschränken. Dies bestand daraus, dass die Klausel, welche das Verhalten auf reines Reviewen beschränkten, entfernt wurde. Desweiteren wurden die überarbeiteten TwinCat-Instructions in das Übersicht-Repository eingearbeitet. Somit kann nun der Ablauf aus Abbildung 4 verwendet werden, um die Coding-Conventions und Copilot-Instructions zu verteilen.



Testen Branching und Code-Review

Um die praxisnahe Integration der Code-Quality-Assurance-Protokolle zu bewerten, wurden diese in ein Projekt eingebracht, das sich noch in der Vorinbetriebnahme befand. Hierfür wurde mit den Projektprogrammierern David Kromer und Alexandru Trusi ein Termin geplant, in dem die zu implementierenden Methoden besprochen wurden. Insbesondere die Branching-Strategien erhielten positives Feedback, da beide Programmierer die in Abschnitt 2 beschriebenen Herausforderungen bereits aus der Praxis kannten.

Alexandru brachte zudem den Vorschlag ein, ergänzend zu Feature-Branched pro Entwickler einen eigenen Develop-Branch zu verwenden. Darin kann zunächst unabhängig gearbeitet werden, bevor stabile Zwischenstände per Pull Request in develop überführt werden. Zusätzlich können nutzerspezifische Feature-Branched für die Umsetzung konkreter Funktionen eingesetzt werden. Die Benennung dieser Branches könnte beispielsweise wie folgt lauten:

- user/develop
- user/feature/NeuesFeature

Beide Programmierer signalisierten außerdem Offenheit gegenüber Copilot, sofern die Reviews konsistent und verlässlich arbeiten.

Bei einer Abstimmung mit Robin Hettel zu den Ergebnissen dieses Meetings merkte dieser an, dass man anstatt von Nutzereigenen Branches eher Bereiche der Anlage als eigenen Branch anlegen sollte. Dies soll den Nutzen haben, dass sich Programmierer Bereiche „reservieren“ können, um ungestört an diesen zu Arbeiten. Das würde außerdem dazu beitragen, dass man nicht, nur für Anlagenprogrammierung, eigene

Richtlinien formulieren muss, sondern einen Unternehmensweiten Standard nutzen kann.

6.1 Umsetzung

6.1.1 Branching Strategien

Die Branching-Strategie wurde auf Basis des angepassten Git-Flow-Modells umgesetzt. Dabei wurden die Verantwortlichkeiten der Branches klar definiert und der Merge-Prozess über Pull Requests standardisiert. Ergänzend wurden Benennungsregeln für Feature-, Bugfix- und Release-Banches festgelegt, um Nachvollziehbarkeit und einheitliche Abläufe sicherzustellen.

6.1.2 Copilot Code-Review

Das Copilot-gestützte Code-Review wurde analog zum Test-Repository in den Projektkontext überführt. Dabei werden Pull Requests auf main automatisch gegen die hinterlegten Coding-Conventions geprüft. Die Ergebnisse fließen als strukturierte Review-Kommentare in den Freigabeprozess ein und unterstützen die frühzeitige Erkennung von Abweichungen.

Das Code-Review wird bei jedem Pull Request auf main ausgelöst, um die Einhaltung der Coding-Conventions systematisch zu prüfen.



Implementierung ZenHub

Zenhub ist ein webbasiertes Projektmanagement-Werkzeug, das speziell für Softwareentwicklungsprozesse entwickelt wurde und sich nahtlos in GitHub integriert. Es ermöglicht Teams, Entwicklungsaufgaben direkt dort zu planen, zu priorisieren und zu verfolgen, wo auch der Code entsteht. Durch Funktionen wie Kanban-Boards, Epics, Roadmaps und automatisierte Workflows unterstützt Zenhub eine transparente und kollaborative Arbeitsweise. Besonders in agilen Entwicklungsumgebungen trägt das Tool dazu bei, Kommunikationsaufwand zu reduzieren, Arbeitsabläufe zu strukturieren und datenbasierte Entscheidungen zu erleichtern. Damit bietet Zenhub eine zentrale Plattform, um komplexe Softwareprojekte effizient zu koordinieren und deren Fortschritt nachvollziehbar zu dokumentieren.

7.1 Aktueller Stand bei **E-SW**

Im Moment wird ZenHub bei der **E-SW** verwendet um

8

Fazit

A Literatur

B Glossar

E-SW: Entwicklung Software - Abteilung bei der fpt Systems GmbH, welche die Entwicklung neuer FlexOp Versionen und anderer Hochsprachenprojekten übernimmt [1](#), [5](#), [7](#), [7](#), [7](#), [8](#), [13](#), [13](#), [25](#), [25](#)

FlexOp: Firmeneigene Nutzeroberfläche zu Steuerung und Verwaltung von Anlagen [5](#), [7](#)

O-SW: Operations Software - Abteilung bei der fpt Systems GmbH, welche die Programmierung von SPS Projekten übernimmt [5](#), [5](#), [5](#), [7](#), [7](#), [7](#), [7](#), [8](#), [10](#), [11](#), [12](#), [17](#)

TwinCat: Aktuelle Version der Entwicklungsumgebung für Beckhoff SPS [6](#), [18](#)

fpt Systems GmbH: Tochterfirma der FPT Robotik GmbH & Co. KG, welche für die Softwareentwicklung verantwortlich ist. [7](#)

Sperrvermerk

Die vorliegende Arbeit mit dem Titel

Konzeption und Implementierung eines GitHub-basierten Branching-, Review- und Projektmanagementprozesses mit KI-gestützter Code-Compliance

enthält unternehmensinterne bzw. vertrauliche Informationen der fpt Systems GmbH, ist deshalb mit einem Sperrvermerk versehen und wird ausschließlich zu Prüfungszwecken im Studiengang Elektrotechnik der Duale Hochschule Baden-Württemberg Ravensburg Campus Friedrichshafen vorgelegt.

Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anders lautende Genehmigung der Ausbildungsstätte (fpt Systems GmbH) vorliegt.

Amtzell 23.04.2026



Florian Germann

Selbstständigkeitserklärung

Gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden-Württemberg vom 29.09.2017. Ich versichere hiermit, dass ich meine Arbeit mit dem Thema:

Konzeption und Implementierung eines GitHub-basierten Branching-, Review- und Projektmanagementprozesses mit KI-gestützter Code-Compliance

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass alle eingereichten Fassungen übereinstimmen.

Amtzell 23.04.2026



Florian Germann